



# Homework #2



|     |            |
|-----|------------|
| 과목명 | 운영 체제      |
| 교수명 | 강동현 교수님    |
| 제출일 | 2026.05.16 |
| 전 공 | 컴퓨터AI학부    |
| 학 번 | 2022112446 |
| 성 명 | 황정민        |

## 1. 코드 분석 - 공통 구조

### 변수 설정

```
#define NUM_THREADS 4
#define ITERATIONS 100000
#define FIRST_PHASE_TRIES 10 //페이지 1에서 10번 spin

volatile int lock_flag = 0;
long long counter = 0;

long long lock_count = 0;
long long unlock_count = 0;
long long total_lock_wait_ns = 0;
long long sleep_count = 0;
```

NUM\_THREADS는 생성할 스레드 수이고, ITERATIONS는 각 스레드가 counter를 증가시키는 반복 횟수이다. counter는 모든 스레드가 함께 접근하는 공유 변수이다. lock\_count, unlock\_count, total\_lock\_wait\_ns는 각각 lock 수행 횟수, unlock 수행 횟수, lock 대기 시간을 측정하기 위한 변수이다.

### get\_ns\_between() - 시간 측정 함수

```
//두 time 사이의 시간 ns 단위로 계산
long long get_ns_between(struct timespec start, struct timespec end)
{
    return (end.tv_sec - start.tv_sec) * 1000000000LL + (end.tv_nsec - start.tv_nsec);
}
```

본 과제에서는 성능 분석 지표로 CPU 사용률, 전체 수행 시간, Lock 대기 시간을 요구했다. timespec 구조체에 저장되는 시간은 초 단위인 tv\_sec와 나노초 단위인 tv\_nsec로 나누어 저장되기 때문에 초 단위를 나노초로 변환한 뒤, 나노초 차이와 더해 전체 시간 차이를 ns 단위로 반환하도록 했다.

### CPU 사용 시간 계산

```
//timeval 구조체 값을 초 단위로 변환
double timeval_to_sec(struct timeval tv)
{
    return (double)tv.tv_sec + (double)tv.tv_usec / 1000000.0;
}

//CPU 사용 시간 계산
double get_cpu_time(struct rusage start, struct rusage end)
{
    double start_cpu = timeval_to_sec(start.ru_utime) + timeval_to_sec(start.ru_stime);
    double end_cpu = timeval_to_sec(end.ru_utime) + timeval_to_sec(end.ru_stime);
    return end_cpu - start_cpu; //사용자 및 커널 영역에서의 time으로 계산
```

getrusage()는 프로세스가 사용한 CPU 시간을 가져온다. 여기서 ru\_utime은 사용자 영역에서 사용한 시간이고, ru\_stime은 커널 영역에서 사용한 시간이다. 두 값을 더해 전체 CPU 사용 시간을 계산했다. 전체 CPU 사용 시간은 이후 CPU 사용률을 main()에서 CPU 사용 시간 / 전체 수행 시간 \* 100으로 계산할 때 사용하기 때문에 본 함수를 구현했다.

## worker() – 공유 자원 접근 함수

```
//counter를 ITERATIONS번 증가시키는 함수
void *worker(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        lock();
        //critical section
        counter++;
        unlock();
    }

    return NULL;
}
```

worker()는 각 스레드가 실행하는 함수이다. pthread\_create()로 생성된 스레드는 이 함수를 실행하고, 각 스레드는 ITERATIONS번 반복하면서 counter를 증가시킨다. counter++는 공유 변수에 접근하는 critical section 이므로, 앞뒤에 lock()과 unlock()을 배치하여 한 번에 하나의 스레드만 접근하도록 했다.

## main() – 변수 선언 및 critical section 접근 로직

```
int main(void)
{
    pthread_t threads[NUM_THREADS];
    struct timespec start_time, end_time;
    struct rusage cpu_start, cpu_end;

    counter = 0;
    lock_flag = 0;
    lock_count = 0;
    unlock_count = 0;
    total_lock_wait_ns = 0;
    sleep_count = 0;

    //전체 수행 시간과 CPU 사용 시간을 측정하기 위한 측정 시작 시점
    getrusage(RUSAGE_SELF, &cpu_start);
    clock_gettime(CLOCK_MONOTONIC, &start_time);

    //스레드 생성
    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, worker, NULL) != 0) {
            perror("pthread_create");
            return EXIT_FAILURE;
        }
    }

    //Join
    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_join(threads[i], NULL) != 0) {
            perror("pthread_join");
            return EXIT_FAILURE;
        }
    }

    //측정 종료 시점
    clock_gettime(CLOCK_MONOTONIC, &end_time);
    getrusage(RUSAGE_SELF, &cpu_end);
}
```

우선 성능 분석 지표 계산을 위한 변수를 정의했다. Two-phase 랙 같은 경우 sleep\_count를 별도로 추가했다. 이후 스레드 생성과 조인 로직 앞뒤로 getrusage()과 clock\_gettime()을 통해 해당 시점 시간을 측정했다. 스레드 생성 및 조인은 반복문을 통해 구현해줬다.

## main() – 분석 및 결과 출력

```
//분석값 계산
double total_runtime = (double)get_ns_between(start_time, end_time) / 1000000000.0;
double cpu_sec = get_cpu_time(cpu_start, cpu_end);
double cpu_usage = total_runtime > 0.0 ? (cpu_sec / total_runtime) * 100.0 : 0.0;
double total_wait_sec = (double)total_lock_wait_ns / 1000000000.0;

//분석 결과 출력
printf("Lock Type: TWO\n");
printf("NUM_THREADS: %d\n", NUM_THREADS);
printf("Counter: %ld\n", counter);
printf("Total Runtime(sec): %.9f\n", total_runtime);
printf("CPU Usage(%%): %.2f\n", cpu_usage);
printf("Lock Count: %ld\n", lock_count);
printf("Unlock Count: %ld\n", unlock_count);
printf("Total Lock Wait Time(sec): %.9f\n", total_wait_sec);
printf("Sleep Count: %ld\n", sleep_count);

return EXIT_SUCCESS;
```

전체 스레드가 종료된 뒤, 위에서 구한 측정한 시작 시점과 종료 시점 시간의 차이를 이용하여 전체 수행 시간을 계산했다. lock 로직 내부에서 구한 분석 변수들의 값을 위에서 정의한 get\_ns\_between(), get\_cpu\_time() 함수를 통해 구하고 출력해줬다.

## 2. 코드 분석 – TAS lock

### TestAndSet()

```
//기존 값을 old로 반환하고, 동시에 new 값을 저장
int TestAndSet(volatile int *ptr, int new)
{
    int old = __sync_lock_test_and_set(ptr, new);
    return old;
}
```

TAS lock 구현을 위해 만든 함수이다. 이 함수는 ptr이 가리키는 기존 값을 old로 저장하고, 동시에 해당 위치에 new 값을 저장한다. 본 코드에서는 lock\_flag에 대해 이 연산을 수행하며, lock\_flag가 0이면 lock이 비어 있는 상태, 1이면 이미 사용 중인 상태로 해석한다.

### lock()

```
//TAS 기반 spin lock
void lock(void)
{
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    //old==1 -> 이미 lock이 걸린 상태이므로 계속 spin
    while (TestAndSet(&lock_flag, 1) == 1) {
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    __sync_fetch_and_add(&lock_count, 1);
    // lock 진입 시점부터 실제 획득 시점까지의 시간을 누적 저장
    __sync_fetch_and_add(&total_lock_wait_ns, get_ns_between(start, end));
}
```

lock()에서는 TestAndSet(&lock\_flag, 1)을 반복 호출한다. 반환값이 1이면 이미 다른 스레드가 lock을 잡고 있는 상태이므로 계속 spin하며 대기한다. 반환값이 0이면 기존 lock이 비어 있었다는 의미이고, 동시에 lock\_flag를 1로 바꾸었으므로 lock 획득에 성공한다.

추가로 분석 지표 계산을 위해 lock 진입 시점부터 실제 획득 시점까지의 시간을 누적하여 저장했다.

### unlock()

```
// lock을 해제하고 unlock 횟수를 기록
void unlock(void)
{
    __sync_fetch_and_add(&unlock_count, 1);
    __sync_lock_release(&lock_flag);
}
```

unlock()에서는 unlock 수행 횟수를 증가시킨 뒤 lock\_flag를 0으로 되돌린다.

### 3. 코드 분석 - CAS lock

#### CompareAndSwap()

```
//expected와 같으면 new로 바꾸고, 실제 기존 값을 actual로 반환
int CompareAndSwap(volatile int *ptr, int expected, int new)
{
    int actual = __sync_val_compare_and_swap(ptr, expected, new);
    return actual;
}
```

CompareAndSwap()은 현재 값이 expected와 같을 때만 new로 변경하는 원자적인 연산을 하는 함수이다. 이 함수는 실제 기존 값을 actual로 반환한다. 본 코드에서는 lock\_flag가 0일 때만 1로 변경하여 lock을 획득하도록 구현했다.

#### lock()

```
//CAS 기반 spin lock
void lock(void)
{
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    //actual == 0 -> lock 획득 성공, 1이면 이미 lock이 잡힌 상태
    while (CompareAndSwap(&lock_flag, 0, 1) != 0) {
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    __sync_fetch_and_add(&lock_count, 1);
    //lock 진입 시점부터 실제 획득 시점까지의 시간을 누적 저장
    __sync_fetch_and_add(&total_lock_wait_ns, get_ns_between(start, end));
}
```

lock()에서는 lock\_flag가 0이면 1로 바꾸며 lock 획득에 성공한다. 이때 반환값은 기존 값인 0이므로 반복문을 빠져나간다. 반대로 반환값이 1이면 이미 다른 스레드가 lock을 잡고 있는 상태이므로 계속 spin한다.

TAS와 마찬가지로 분석 지표 계산을 위해 lock 진입 시점부터 실제 획득 시점까지의 시간을 누적하여 저장했다.

#### unlock()

```
//lock 해제하고 unlock 횟수를 기록
void unlock(void)
{
    __sync_fetch_and_add(&unlock_count, 1);
    __sync_lock_release(&lock_flag);
}
```

unlock 구조는 TAS와 동일하다. critical section이 끝나면 lock\_flag를 0으로 되돌려 다른 스레드가 lock을 획득할 수 있게 한다. 분석을 위해 unlock 수행 횟수도 1 증가시킨다.

## 4. 코드 분석 - Two-phase lock

### FIRST\_PHASE\_TRIES

```
#define FIRST_PHASE_TRIES 10 //first phase에서 CAS spin을 시도하는 횟수
```

FIRST\_PHASE\_TRIES를 10으로 설정하여 first phase에서 CAS 기반 lock 획득을 10번까지 시도하도록 구현했다.

### 보조 mutex, condition variable

```
//condition variable 대기/깨우기 흐름을 위한 보조 mutex와 condition variable
pthread_mutex_t wait_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t wait_cond = PTHREAD_COND_INITIALIZER;
```

Second phase에서 sleep과 wakeup 흐름을 구현하기 위해 condition variable과 보조 mutex를 추가했다. 여기서 wait\_mutex는 counter를 보호하는 lock이 아니라, pthread\_cond\_wait()와 pthread\_cond\_signal()을 안전하게 사용하기 위한 보조 mutex다. wait\_cond는 lock을 기다리는 스레드를 sleep 상태로 두었다가, unlock 시점에 다시 깨우기 위해 사용했다.

### spin - CAS로 수행

```
//CAS로 spin
int CompareAndSwap(volatile int *ptr, int expected, int new)
{
    int actual = __sync_val_compare_and_swap(ptr, expected, new);
    return actual;
}
```

Two-phase lock의 first phase에서는 CAS 방식으로 lock 획득을 10번 시도하도록 구현했다. 따라서 앞서 구현한 CompareAndSwap()을 그대로 사용했다.

### lock()

```
//Two-phase lock: 먼저 spin을 시도하고, 실패하면 condition variable에서 대기
void lock(void)
{
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    while (1) {
        //First phase: CompareAndSwap으로 lock 획득을 10번 시도
        for (int i = 0; i < FIRST_PHASE_TRIES; i++) {
            if (CompareAndSwap(&lock_flag, 0, 1) == 0) {
                clock_gettime(CLOCK_MONOTONIC, &end);
                __sync_fetch_and_add(&lock_count, 1);
                //lock 진입 시점부터 현재 획득 시점까지의 시간을 누적 저장
                __sync_fetch_and_add(&total_lock_wait_ns, get_ns_between(start, end));
                return;
            }
        }

        //second phase: lock이 풀릴 때까지 sleep 상태로 대기
        __sync_fetch_and_add(&sleep_count, 1);
        pthread_mutex_lock(&wait_mutex);
        while (lock_flag == 1) {
            pthread_cond_wait(&wait_cond, &wait_mutex);
        }
        pthread_mutex_unlock(&wait_mutex);
    }
}
```

Two-phase lock의 first phase에서는 CAS로 lock 획득을 10번 시도한다. 10번 안에 lock\_flag가 0인 순간을 만나면 1로 변경하고 lock 획득에 성공한다. 성공 시 lock 수행 횟수와 lock 대기 시간을 기록한 뒤 함수를 종료한다.

second phase에서는 sleep\_count를 증가시켜 sleep 상태로 들어간 횟수를 기록한다. 이후 pthread\_cond\_wait()를 호출하여 스레드를 sleep 상태로 전환한다. pthread\_cond\_wait()는 내부적으로 mutex를 잠시 해제한 뒤 sleep 상태로 들어가며, 이후 pthread\_cond\_signal()에 의해 깨어나면 다시 mutex를 획득한 뒤 실행하도록 한다.

또한 while(lock\_flag == 1) 조건을 사용한 이유는 spurious wakeup 때문이다. Condition variable은 signal 없이도 깨어날 수 있기 때문에, 깨어난 이후에도 실제로 lock이 풀렸는지 다시 확인하도록 구현했다.

## unlock()

```
//lock 해제하고 unlock 횟수를 기록
void unlock(void)
{
    __sync_fetch_and_add(&unlock_count, 1);
    __sync_lock_release(&lock_flag);

    //lock이 풀렸으므로 대기 중인 스레드 하나를 깨움
    pthread_mutex_lock(&wait_mutex);
    pthread_cond_signal(&wait_cond);
    pthread_mutex_unlock(&wait_mutex);
}
```

unlock()에서는 먼저 unlock 수행 횟수를 증가시키고, \_\_sync\_lock\_release()를 통해 lock\_flag를 0으로 되돌린다. 이후 pthread\_cond\_signal()을 호출하여 second phase에서 sleep 상태로 대기 중인 스레드 하나를 깨운다.

## 5. 결과 사진

NUM\_THREADS: 1

```
Lock Type: TAS
NUM_THREADS: 1
Counter: 100000
Total Runtime(sec): 0.011871938
CPU Usage(%): 71.82
Lock Count: 100000
Unlock Count: 100000
Total Lock Wait Time(sec): 0.004425687
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./cas
Lock Type: CAS
NUM_THREADS: 1
Counter: 100000
Total Runtime(sec): 0.011369165
CPU Usage(%): 98.39
Lock Count: 100000
Unlock Count: 100000
Total Lock Wait Time(sec): 0.004377422
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./two
Lock Type: TWO
NUM_THREADS: 1
Counter: 100000
Total Runtime(sec): 0.014478853
CPU Usage(%): 92.10
Lock Count: 100000
Unlock Count: 100000
Total Lock Wait Time(sec): 0.004556032
Sleep Count: 0
```

NUM\_THREADS: 2

```
Lock Type: TAS
NUM_THREADS: 2
Counter: 200000
Total Runtime(sec): 0.054980126
CPU Usage(%): 194.52
Lock Count: 200000
Unlock Count: 200000
Total Lock Wait Time(sec): 0.063201424
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./cas
Lock Type: CAS
NUM_THREADS: 2
Counter: 200000
Total Runtime(sec): 0.056321752
CPU Usage(%): 189.64
Lock Count: 200000
Unlock Count: 200000
Total Lock Wait Time(sec): 0.065175924
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./two
Lock Type: TWO
NUM_THREADS: 2
Counter: 200000
Total Runtime(sec): 0.052598570
CPU Usage(%): 177.18
Lock Count: 200000
Unlock Count: 200000
Total Lock Wait Time(sec): 0.053009040
Sleep Count: 10777
```



## NUM\_THREADS: 4

```
Lock Type: TAS
NUM_THREADS: 4
Counter: 400000
Total Runtime(sec): 0.206393497
CPU Usage(%): 393.05
Lock Count: 400000
Unlock Count: 400000
Total Lock Wait Time(sec): 0.637082069
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./cas
Lock Type: CAS
NUM_THREADS: 4
Counter: 400000
Total Runtime(sec): 0.198842519
CPU Usage(%): 388.11
Lock Count: 400000
Unlock Count: 400000
Total Lock Wait Time(sec): 0.604886000
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./two
Lock Type: TWO
NUM_THREADS: 4
Counter: 400000
Total Runtime(sec): 0.126900009
CPU Usage(%): 248.53
Lock Count: 400000
Unlock Count: 400000
Total Lock Wait Time(sec): 0.196873197
Sleep Count: 21510
```

## NUM\_THREADS: 8

```
^[[ALock Type: TAS
NUM_THREADS: 8
Counter: 800000
Total Runtime(sec): 0.564276616
CPU Usage(%): 749.25
Lock Count: 800000
Unlock Count: 800000
Total Lock Wait Time(sec): 3.800205152
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./cas
Lock Type: CAS
NUM_THREADS: 8
Counter: 800000
Total Runtime(sec): 0.519917162
CPU Usage(%): 653.51
Lock Count: 800000
Unlock Count: 800000
Total Lock Wait Time(sec): 3.020500211
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./two
Lock Type: TWO
NUM_THREADS: 8
Counter: 800000
Total Runtime(sec): 0.268727850
CPU Usage(%): 415.40
Lock Count: 800000
Unlock Count: 800000
Total Lock Wait Time(sec): 0.735590891
Sleep Count: 44957
```

## NUM\_THREADS: 16

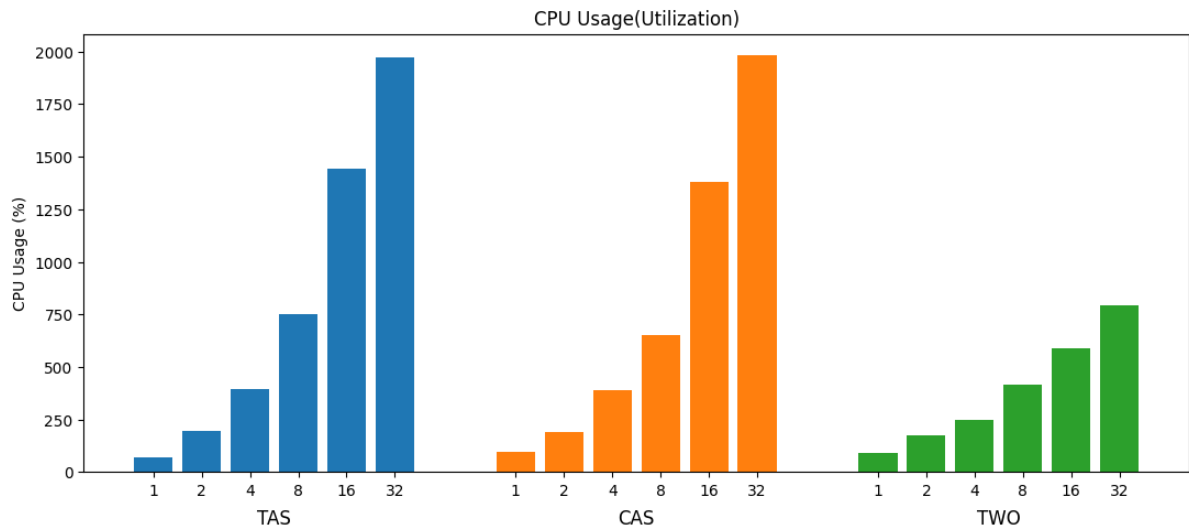
```
Lock Type: TAS
NUM_THREADS: 16
Counter: 1600000
Total Runtime(sec): 2.303417561
CPU Usage(%): 1444.05
Lock Count: 1600000
Unlock Count: 1600000
Total Lock Wait Time(sec): 31.267581268
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./cas
Lock Type: CAS
NUM_THREADS: 16
Counter: 1600000
Total Runtime(sec): 2.024342807
CPU Usage(%): 1380.49
Lock Count: 1600000
Unlock Count: 1600000
Total Lock Wait Time(sec): 26.224260963
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./two
Lock Type: TWO
NUM_THREADS: 16
Counter: 1600000
Total Runtime(sec): 0.714749307
CPU Usage(%): 588.69
Lock Count: 1600000
Unlock Count: 1600000
Total Lock Wait Time(sec): 3.754815487
Sleep Count: 92721
```

## NUM\_THREADS: 32

```
Lock Type: TAS
NUM_THREADS: 32
Counter: 3200000
Total Runtime(sec): 6.457050277
CPU Usage(%): 1974.57
Lock Count: 3200000
Unlock Count: 3200000
Total Lock Wait Time(sec): 165.446214467
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./cas
Lock Type: CAS
NUM_THREADS: 32
Counter: 3200000
Total Runtime(sec): 6.823463844
CPU Usage(%): 1984.18
Lock Count: 3200000
Unlock Count: 3200000
Total Lock Wait Time(sec): 173.318762751
jmhwan@localhost:/mnt/c/Users/jmhwa/OneDrive/바탕 화면/학교 수업/운영체제/운체 과제/Homework2$ ./two
Lock Type: TWO
NUM_THREADS: 32
Counter: 3200000
Total Runtime(sec): 1.621171579
CPU Usage(%): 791.87
Lock Count: 3200000
Unlock Count: 3200000
Total Lock Wait Time(sec): 15.996279103
Sleep Count: 178607
```

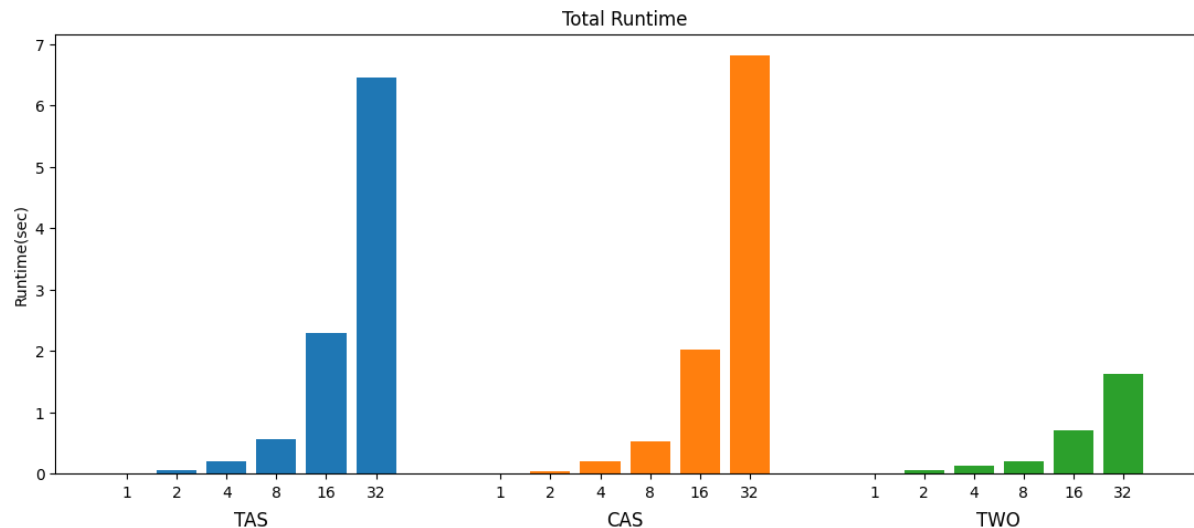
## 6. 결과 그래프 및 결과 요약

### CPU Usage



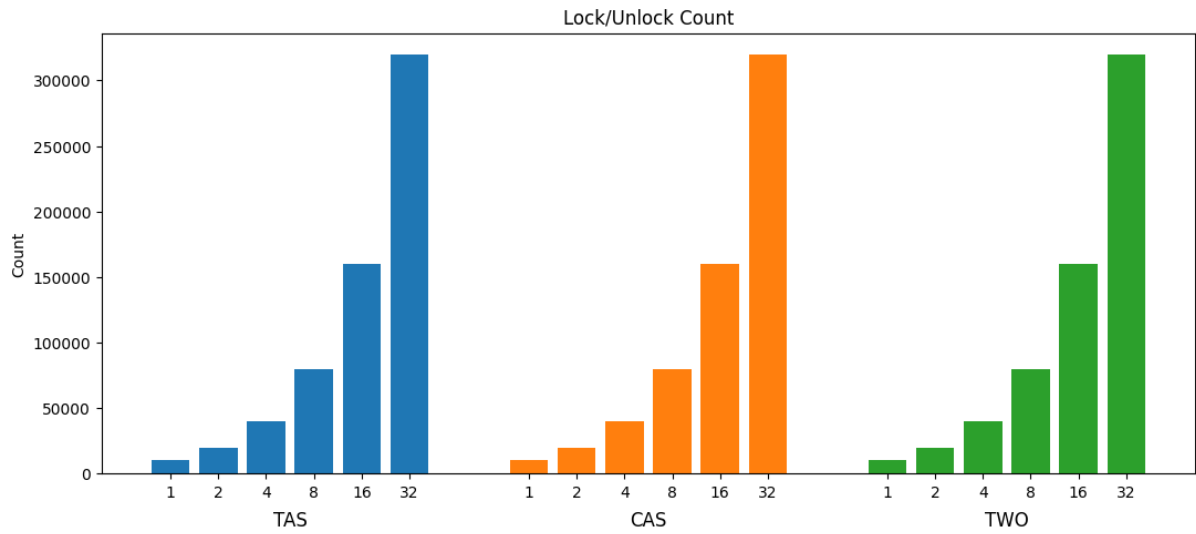
TAS와 CAS는 스레드 수가 증가할수록 CPU 사용률이 급격하게 증가했다. 특히 16~32 thread 구간에서는 두 방식 모두 1400~2000% 수준까지 상승하며 매우 높은 CPU 사용량을 보였다. 반면 Two-phase lock은 thread 수가 증가해도 CPU 사용률 증가 폭이 상대적으로 완만했으며, 32 thread에서도 약 800% 수준으로 TAS/CAS 보다 훨씬 낮게 유지되었다. 실제 결과 기준 가장 CPU 낭비가 적은 방식은 Two-phase lock이었다.

### Total Runtime



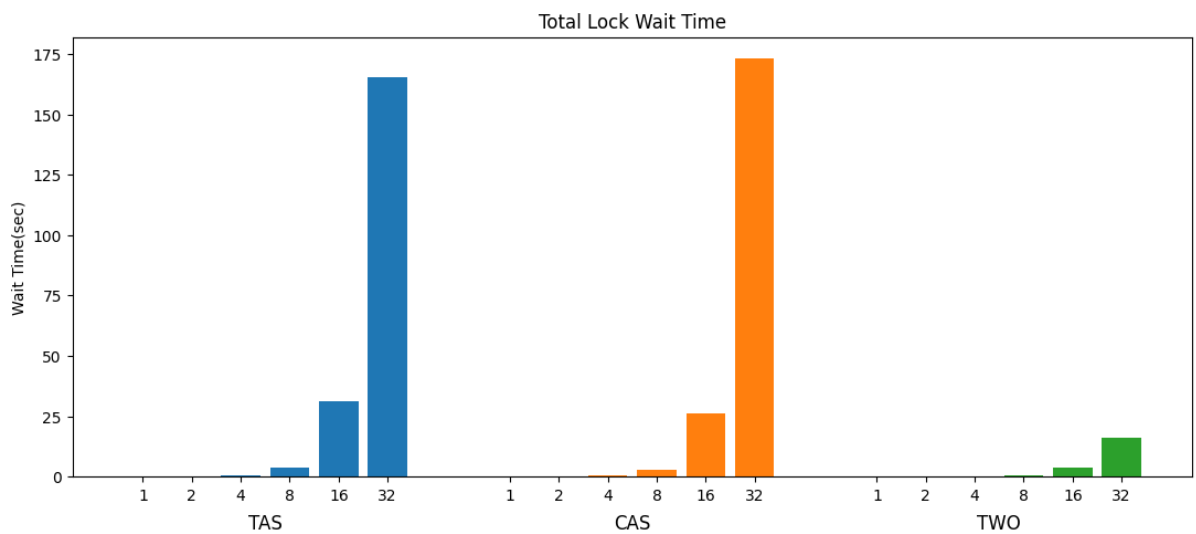
초반에 스레드 수가 적은 구간에서는 세 방식의 runtime 차이가 크지 않았다. 그러나 thread 수가 증가할수록 TAS와 CAS의 runtime이 급격하게 증가했고, 특히 32 thread에서는 6초 이상까지 증가했다. 반면 Two-phase lock은 같은 환경에서도 약 1.6초 수준으로 유지되며 가장 짧은 runtime을 보였다. TAS와 CAS를 비교하면 후반부에서는 오히려 TAS가 CAS보다 약간 더 짧게 측정되었다.

## Lock/Unlock Count



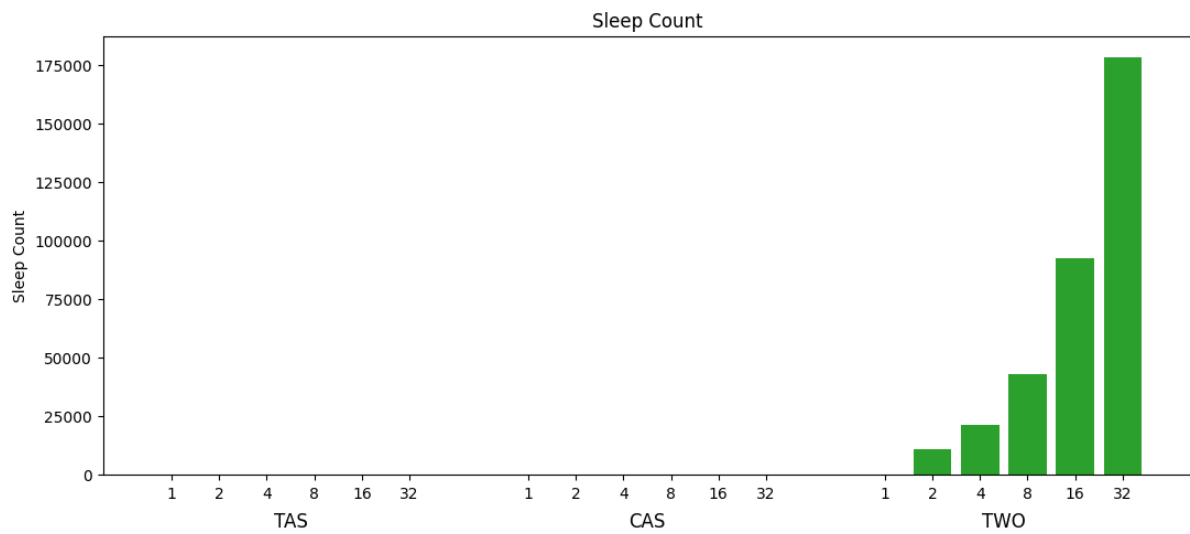
세 방식 모두 각 스레드가 ITERATIONS만큼 critical section에 진입하므로 lock/unlock count는 스레드 수와 ITERATIONS에 비례하여 증가했다.

## Total Lock Waiting Time



Lock wait time은 thread 수 증가에 따라 모든 방식에서 증가했다. TAS와 CAS는 특히 16~32 thread 구간에서 wait time이 매우 크게 증가했으며, 32 thread에서는 160초 이상 수준까지 증가했다. 반면 Two-phase lock은 같은 환경에서도 약 16초 수준으로 상대적으로 매우 낮게 유지되었다. 실제 결과 기준 wait time 측면에서는 Two-phase lock이 가장 안정적인 성능을 보였다.

## Sleep Count



sleep은 Two-phase lock에서만 측정되었으며, thread 수 증가에 따라 지속적으로 증가했다. 이는 thread 수가 많아질수록 first phase의 CAS spin만으로 lock 획득에 실패하는 경우가 많아졌다는 의미로 해석 가능하다. 특히 16~32 thread 구간에서 sleep count 증가 폭이 크게 나타났으며, 그만큼 second phase로 진입하는 thread 수가 많아졌음을 확인할 수 있다.

## 7. 결과 분석

### 1. CAS와 TAS의 CPU 사용률 차이가 예상보다 크지 않았던 이유

CAS는 예상 값 비교를 통해 조건부 갱신을 하여 메모리에 1을 갱신을 하지 않기 때문에 TAS보다 효율적일 것이라고 예상했지만, 실제 결과에서는 CPU 사용률 차이가 생각보다 크지 않았다. 이는 두 방식 모두 결국 lock을 얻을 때까지 계속 spin하기 때문이라고 생각한다. CAS 또한 lock 획득 실패 시 비교 및 재시도를 반복하면서 CPU를 지속적으로 사용하게 되었고, 결과적으로 TAS와 비슷한 수준의 CPU 사용량 증가가 발생한 것으로 보인다. 즉, CAS가 비교 기반 구조라는 차이는 존재했지만, 실제로 돌려봤을 때는 두 방식 모두 CPU를 양보하지 않는 spin lock이라는 공통점의 영향이 더 크게 나타났다고 생각한다.

### 2. 높은 Thread 환경에서 TAS의 Runtime이 오히려 더 짧게 나온 이유

실행 결과를 보면 CPU 사용량은 CAS가 TAS보다 약간 낮은 경우가 많았지만, 후반부 runtime은 오히려 TAS가 더 짧게 측정되었다. 처음에는 메모리 갱신 오버헤드를 생각해서 CAS가 TAS보다 항상 더 좋은 결과를 낼 것이라고 생각했지만 실제 결과는 달랐다. 이번 과제의 critical section은 사실상 counter를 1 증가시키는 매우 짧은 작업이었기 때문에, 실제 작업보다 lock 처리 비용 자체의 비중이 더 커졌다고 생각한다. CAS는 compare 기반 retry 과정을 반복하면서 오버헤드가 누적되었고, 반대로 TAS는 구조 자체가 단순하기 때문에 높은 CPU 사용량에도 불구하고 오히려 runtime은 더 짧게 나온 것으로 보인다. 이를 통해 CPU 효율성과 실제 수행 속도가 반드시 비례하는 것은 아니라고 분석했다.

### 3. Two-phase Lock의 CPU 효율은 높았지만 Runtime 차이는 상대적으로 작았던 이유

Two-phase lock은 CPU 사용률과 wait time 측면에서 가장 안정적인 결과를 보였다. 특히 thread 수가 많아질수록 TAS와 CAS는 spin으로 인한 CPU 낭비가 매우 커졌지만, Two-phase lock은 일정 횟수 spin 이후 sleep 상태로 전환되기 때문에 CPU 사용량 증가 폭이 상대적으로 작았다. 즉, lock 경쟁이 심한 환경일수록 spin lock에 비해 CPU 낭비하는 시간을 크게 줄일 수 있다는 점에서 Two-phase 방식의 장점이 크게 나타났다고 생각한다. 실제 결과에서도 sleep count가 thread 수 증가에 따라 크게 증가했는데, 이는 많은 thread가 first phase의 spin만으로는 lock을 얻지 못하고 second phase로 넘어갔다는 의미로 해석할 수 있었다.

#### 3-1. Two-phase lock의 맹점

하지만 모든 상황에서 Two-phase lock이 무조건적으로 좋은 것은 아니라고 생각했다. sleep과 wakeup 과정에서는 scheduler 개입과 문맥 교환이 발생할 수 있는데, 이러한 비용 또한 결코 작지 않다. 특히 이번 구현에서는 CPU 사용률, runtime, wait time 등은 측정했지만 실제 문맥 교환 횟수나 scheduler overhead 자체는 직접 측정하지 못했다. 따라서 결과 그래프만 보면 Two-phase lock이 가장 효율적인 방식처럼 보일 수 있지만, 실제 시스템 환경에서는 문맥 교환 비용까지 함께 고려해야 한다.

### 4. Lock 방식에 따른 성능 차이와 전체 결과 해석

전체적으로 이번 실험을 통해 단순히 runtime만으로 lock 성능을 판단할 수는 없다는 점을 확인할 수 있었다. TAS는 CPU 사용률은 매우 높았지만 단순한 구조 덕분에 짧은 runtime을 보였고, CAS는 비교 기반 구조로 인해 동시 접근이 심한 환경에서는 예상보다 큰 오버헤드가 발생했다. 반면 Two-phase lock은 CPU 효율성 측면에서는 가장 좋은 결과를 보였지만, sleep/wakeup 기반 구조 특성상 문맥 교환 비용이라는 또 다른

오버헤드를 가졌다. 최종적으로 이번 실행을 돌려보며 안정성 측면에서는 예상 값 비교 기반인 CAS, 단순 로직에서는 TAS, 스레드가 많은 경우에는 spin으로 인한 CPU 낭비를 줄이기 위해 Two-phase lock을 쓰되, 문맥 교환 비용도 고려하여 적절한 trade-off를 해야 한다고 생각했다.